Trading Mantissa Bits for Watts: Software Mantissa Zeroing under Module Power Caps on the NVIDIA GH200

Koying Chen

***Abstract*—We study a software knob exploiting data-dependent GPU power—zeroing unneeded low-order mantissa bits of FP64, BF16 and FP8 operands—on 14 NVIDIA GH200 nodes under module power caps, with paired measurements and an equivalent-power metric (the extra cap the unmodified program needs). Under a binding cap, every tested HPC program that truncates internally and checks its results gains. At 400–800 W, tuned 8-node HPL keeping 36 significand bits ($\approx 100\times$ residual margin) runs 4.0–11.0% faster, worth 39–127 W per node, and HPL-MxP keeping 3 bits 1.8–12.7% on a diagonally dominant test matrix; at 450–550 W Quantum ESPRESSO, with bits tied to its Davidson tolerance, runs 2.4–4.2% faster (partly from fewer iterations) with all printed energy digits unchanged; at 500–700 W DF-MP2 keeping 20–32 bits runs 4.9–8.3% faster, and a wide-range Gaussian-process likelihood keeping 28 bits 7.4–8.2%, where FP32 fails and INT8 emulation is slower. At the default 1000 W cap HPL is still 3.9% faster and uses 7.5% less energy per solve. Fill controls attribute 62–97% of the saving to bits that stop toggling. Safe bit counts need validation across problem sizes; the crossover with INT8 emulation of FP64 moves with matrix shape and cap. With wide per-row exponent spans, fixed-width INT8 emulation can silently fail: at 36 bits it shifts a Gaussian-basis DFT energy by 52 $\mathrm{mE_h}$, whereas rounding to 36 bits errs by $10^{-6}\,\mathrm{E_h}$. All-FP32 or 3xTF32 GEMMs fail on long inner products, and in Hopper AI inference FP8 already captures most of the benefit.**

***Index Terms*—power capping, energy efficiency, floating-point precision, mantissa truncation, GPU, mixed precision, HPL**

I. Introduction

Power is becoming the ceiling on delivered performance: data centers cap rack power, student cluster competitions cap whole clusters, and the NVIDIA GH200 lets users cap the entire module. Under a cap the power manager lowers the clock until the module fits, so a program's speed depends on how much energy its operations consume.

That GPU power depends on input data is known: random operands draw far more power than constant or all-zero ones [1], [2], zeroed low-order bits lower GEMM power [3], the gap between all-zero and random GEMMs widens as the power limit is lowered [4], and hardware can narrow or mask mantissas [5]–[9]. These works use special inputs such as all-zero or constant data (He also swept the power limit, but only for all-zero against random inputs) or require new hardware or simulated energy. To our knowledge this is the first study to truncate part of the mantissa of *real* FP64/BF16 operands in software on a production GPU and measure, on multi-node benchmarks and real HPC applications checked by their own correctness criteria, how much throughput the saved switching power buys under power caps, at what accuracy cost, and where it still pays off against FP8 or INT8 emulation of FP64 [10], [11]. Our contributions are:

1) **Gains on real applications under power caps.** We express the benefit as the extra cap the unmodified program needs to catch up, from GEMM (with paired confidence intervals) to tuned 8-node HPL (36 significand bits: 4.0–11.0% faster at 400–800 W, 39–127 W), HPL-MxP on a diagonally dominant test matrix, Quantum ESPRESSO, power-hungry DF-MP2 (4.9–8.3% at 500–700 W) and a wide-range, power-bound Gaussian-process likelihood (7.4–8.2% at 500–700 W), and check residuals and energies with each code's own criterion (Sections VI-A–VI-E, VII-D).
2) **The mechanism.** Fills of ones or a fixed pattern instead of zeros separate "no toggling" from "value is zero" in tensor-core GEMM, FP64 FMA units and memory traffic, quantifying the toggling hypothesis on GH200 module power (Section III).
3) **Bit counts and failure boundaries.** A rule using only the solver's own tolerance leaves every printed digit of the Quantum ESPRESSO energies unchanged; we test how far HPL (several sizes), PySCF, third-party non-GEMM codes, LoRA fine-tuning and six language models can be truncated, and the role of rounding (Sections VI–VIII).
4) **The boundary with format changes.** Against INT8 emulation (cuBLAS, GEMMul8) and FP32/3xTF32 for FP64, and FP8 for AI, the crossover moves with shape, power cap, accumulation precision and exponent span; with wide per-row exponent spans, fixed-width INT8 emulation can silently err, a risk that cheap per-row span measurements flag beforehand (Sections VII–VIII).

II. Background and Related Work

Input-dependent GPU power. Bhalachandra et al. measured 67% more DGEMM power on random than on constant operands on A100 [1]. Gregersen et al. varied CUTLASS GEMM inputs on V100, A100 and H100, saw power differ by up to almost 40% with zeroed low bits lowering it, and proposed that toggling drives it, but studied only GEMM at sizes that do not throttle [3]. He found all-zero 8192^3 GEMMs on A100 faster than random ones, a speedup that grew from about $1.2\times$ to about $2.3\times$ as the power limit fell from 330 to 150 W, before falling at 100 W [4]. On GH200,

zero-input DGEMM draws 563 W against 898 W for random input [2], while vector FP64 performance shows no significant difference [12]. Constant and random data can differ by about 100 W on H100/H200 [13], and a GTX 580 ALU power model is dominated by toggles, with the number of ones secondary [14]. None of these truncates part of the mantissa of real data or evaluates accuracy or applications.

Power-constrained HPC. Under a power bound, manufacturing variation makes nominally identical processors differ by about 6% in execution time [15] and per-rank DGEMM performance by up to 64%, which variability-aware power allocation mitigates [16]; overprovisioning adds nodes under a fixed system budget [17], and Zeus co-tunes GPU power limit and batch size for DNN training energy [18]. These works distribute power; we lower the energy per operation, so the two compose. GH200 CPU and GPU share module power, and the GPU clock drops near the cap [19]; nvidia-smi samples only about 25% of runtime on A100/H100 [20], which our long steady windows and pairing mitigate.

Hardware masking and data-movement energy. Narrower floating-point multipliers save up to 66% energy per operation [5], and precision has been reduced or masked in mobile-GPU shaders [6], [7], graphics register files [21], a baseband processor [8], GPU shaders [9] and, in simulation, tensor-core register files [22]; rpe and chop emulate low precision with the same bit operations without measuring power [23], [24]. DRAM energy depends on toggling rates [25]–[27], all-zero inputs draw less power as fewer DRAM and L2 bits toggle [28], and compression can increase toggles [29]. We change neither hardware nor data format.

Precision tuning and mixed precision. Precision tuning [30], mixed-precision storage [31] and transprecision computing [32] change types or bytes; zeroing changes neither and composes with them. Reading only leading mantissa segments speeds up memory-bound PageRank by 10–30% by saving bandwidth [33]. Low-precision LU with iterative refinement [34] underlies HPL-MxP [35]. The information content of mantissa bits, estimated from the data [36], could predict how many bits GEMM operands need.

Emulating FP64 on low-precision units. The Ozaki scheme [37] splits a matrix product into exactly computed products of lower-precision slices; it runs on integer units [38], and Ozaki scheme II [10] uses modular arithmetic in GEMMul8, whose speed and energy efficiency Uchino et al. evaluated [11]. cuBLAS's Ozaki-type emulation scales each row of A and column of B by a shared factor, so a row's range is encoded in the mantissa; its automatic mode picks the slice count from the exponent span and falls back to native FP64 beyond a library-defined maximum [39], [40]. Zeroing keeps the original units, so the two can be complementary; we compare them at equal cap and comparable accuracy.

Mixed precision in geostatistics. ExaGeoStat evaluates Gaussian-process likelihoods with an FP64 tile Cholesky [41]; its successors keep diagonal tiles in FP64 and convert others, chosen by distance or norm, to FP32, FP16 or FP8 (1.6× on average with FP32 [42], up to 2.64× adding FP16 [43], 12× with tile low-rank compression [44], 3× on a GH200 [45]), also in a climate emulator [46]. Cao et al. measured power and energy of such factorizations on single GPUs, without caps or a clock analysis [47]. Uniform single precision is insufficient here [43], as we also find; truncation composes with tile conversion (Section VII-D).

Convergence-driven precision. LOBPCG switched between FP32, FP64 and mixed precision by convergence state runs up to 8.5× faster on GPUs [48]; GPU4PySCF can set its number of INT8 emulation slices from the SCF energy change, keeping errors below 10^{-7} hartree ($\mathrm{E_h}$; $1\,\mathrm{E_h} = 2\,\mathrm{Ry} = 627.5\,\mathrm{kcal/mol}$) [49]; RI-MP2 has run with single- and mixed-precision GEMMs [50]. These gain speed from a faster format or unit and do not study power; we stay on FP64 units with bit counts set by the Davidson threshold and gain mainly from clock headroom under a cap (on one system also from fewer Davidson iterations; Section VI-D).

III. Why Fewer Mantissa Bits Save Power

CMOS dynamic power is roughly $\alpha_{\mathrm{sw}} C V^2 f$ with data-dependent switching activity α_{sw}. Under a cap the power manager lowers f (and V) until the module fits, so a lower α_{sw} yields higher clock and throughput, and below the cap lower power. GH200 shows both: at the default 1000 W cap the native FP64 GEMM is still capped 95% of the time at 1822 MHz, while keeping 0 bits it runs at the maximum 1980 MHz and module power falls from 984 to 707 W; under a 700 W cap the SM clock is 1502 MHz native, 1668 MHz with 27 kept bits and 1965 MHz with 0, all at about 694 W.

Toggling or zero? Zeroing both stops the low bits from varying between elements and makes them zero. Fills of all ones or of a fixed nonzero pattern shared by all elements separate the two: neither toggles, neither is zero. At 1000 W all fills run at 1980 MHz, so we compare module power (as the cap holds the native GEMM at 984 W, the share attributed to "no toggling" is conservative); at 500 W every configuration is always capped, so we compare throughput (one node, two runs each; Table I). Merely stopping the toggling captures 68–92% of the benefit in tensor-core GEMM, quantifying the toggling hypothesis [3] on GH200 module power; zeros save somewhat more. With 20 kept bits this fits multiplier structure: low bits of a fixed at one still form partial products $a_i b_j$ that toggle with the kept bits of b, while zeros keep that slice at zero. With 0 kept bits no partial product toggles under any fill, yet zeros still save more than ones (707 versus 774 W), perhaps because exact powers of two align and accumulate more simply (unverified). Lacking gate-level data, both are inferences, consistent with a GTX 580 ALU model in which toggles dominate and the number of ones is secondary [14].

Not just tensor cores. Keeping 0 bits lowers module power at 1000 W from 862 to 553 W in an FP64 CUDA-core multiply-add loop and from 672 to 549 W (-18%) in a STREAM-like copy at 3.33 TB/s, and ones or the fixed pattern keep 62–87% and 86–97% of these savings (Table I): fewer toggles in arithmetic units and on data wires are the main

TABLE I

FILLING THE DISCARDED BITS WITH ZEROS, ONES OR A FIXED PATTERN (TC: TENSOR CORE). 1000 W ROWS GIVE MODULE POWER IN W, 500 W ROWS THROUGHPUT IN TFLOP/S (ONE NODE, TWO RUNS EACH). THE LAST COLUMN IS THE SHARE OF THE ZERO-FILL BENEFIT THAT ONES AND THE FIXED PATTERN RETAIN.

Kernel, kept bits, cap	Native	Pattern	Ones	Zeros	Share
FP64 TC GEMM, 0, 1000 W	984	796	774	707	68–76%
FP64 TC GEMM, 0, 500 W	38.5	47.6	48.3	50.3	77–83%
FP64 TC GEMM, 20, 500 W	38.5	43.1	43.4	44.1	83–87%
BF16 TC GEMM, 0, 500 W	430	542	541	551	92%
FP64 FMA loop, 0, 1000 W	862	671	593	553	62–87%
Memory copy, 0, 1000 W	672	566	553	549	86–97%

cause, also for bandwidth-bound code, consistent with toggle-dependent DRAM energy [25], [27]. An Altera patent's qualitative claim that zeroing low bits at an FPGA floating-point multiplier's input barely reduces power, as the compression tree keeps switching [51], does not contradict this: on a GPU the bits also travel through registers, buses and memory, and even GH200 multipliers save more with zeros than with ones.

IV. METHOD

Mantissa zeroing. For an operand with p fraction bits (52 for FP64, 7 for BF16, 3 for FP8 E4M3) we keep the top k and zero the rest; sign and exponent are untouched. Truncation toward zero is one bit mask; round-to-nearest first adds half a unit of the last kept bit, stochastic rounding a uniform random integer below the weight of the discarded bits. Ties go away from zero in the shim, GEMM microbenchmark, Quantum ESPRESSO, DF-MP2 and Gaussian-process code, and to even in HPL, LoRA weight loading and LLM inference. "Truncate to k bits" covers both: unless stated otherwise, the applications (Quantum ESPRESSO, PySCF, DF-MP2, Gaussian process, ExaGeoStat, third-party codes) and HPL round to nearest, and the GEMM microbenchmarks, crossover sweeps and HPL-MxP truncate toward zero. HPL counts significand bits including the hidden bit, as rocHPL does (36 bits are 35 fraction bits).

Implementation and cost. Each element costs one integer AND (plus an add for rounding), fusable into the kernel that produces the operand, as in our microbenchmarks and HPL-MxP; HPL rounds both operands in a separate kernel before each DGEMM; Quantum ESPRESSO (Davidson vectors), DF-MP2 and the Gaussian-process code zero operands in place; LoRA zeroes the frozen FP8 base weights once after loading. Unmodified software runs under an `LD_PRELOAD` shim that intercepts cuBLAS and CPU GEMM calls, writes zeroed copies of A and B to scratch buffers and can record exponent histograms and per-row spans. Measured times include HPL's extra kernel and the shim's copies.

Error model. For operands uniform on $[-1, 1)$ with A and B zeroed, $\|\hat{C} - C\|_F / \|C\|_F \approx \alpha\, 2^{-k}$ for $C = AB$, with $\alpha \approx 0.27$ (nearest), 0.38 (stochastic) and 0.70 (truncation) for $k \geq 4$, independent of the inner dimension from 256 to 16384. Truncation's error is almost entirely a systematic shrinkage (least-squares slope of $\hat{C}$ against C about $1 - 0.6 \cdot 2^{-k}$), which matters little for solvers that iterate to self-consistency or correct with a residual but hurts where bias accumulates. On real data the 2^{-k} law holds within each GEMM, but α grows with cancellation (nearest: 0.19–0.32 on four Quantum ESPRESSO Si 1000 GEMMs, 0.07 to 9.6 on four gold-surface GEMMs), so it must be measured per application; self-consistent iteration corrects these errors, which is why Quantum ESPRESSO tolerates 20–28 bits even with $\alpha \approx 10$.

Equivalent power saving. If the zeroed program reaches performance $p'(P)$ under cap P, its equivalent power saving $\Delta W(P) = P_{\mathrm{native}}\big(p'(P)\big) - P$ is the extra per-node power the native program needs to match, with $P_{\mathrm{native}}(x)$, the cap at which native reaches x, interpolated linearly along the measured performance–cap curve. If p' exceeds every native point we report a lower bound; if the GPU is not power-bound, the saving appears as lower power at equal speed. As performance is concave in the cap, interpolating between widely spaced caps (e.g., 800 and 1000 W) slightly overestimates ΔW.

Tolerance-driven precision. In plane-wave DFT each Davidson diagonalization gets a threshold $ethr$ that tightens from about 10^{-2} to 10^{-12} during the self-consistent-field (SCF) iteration. We keep $k = \mathrm{clamp}(\lceil -\log_2 ethr \rceil + m,\ k_{\min},\ 52)$ fraction bits (margin m, floor $k_{\min} = 16$). This empirical rule is not a guarantee: $ethr$ is an absolute eigenvalue tolerance (in Ry), not a relative mantissa error, and the margin absorbs the difference; it needs only a tolerance the application already computes.

V. EXPERIMENTAL SETUP

System. Fourteen nodes, each with one GH200 480GB superchip (72-core Grace; Hopper GPU with 96 GB HBM3, maximum SM clock 1980 MHz, FP64 tensor-core peak 67 TFLOP/s [52]), 200 Gb/s InfiniBand, NVHPC 26.5 (CUDA 13.2, cuBLAS 13.4). The module power cap covers GPU, CPU and memory (the GPU alone also has a 900 W limit); the driver accepts 200–1000 W, the datasheet lists 450–1000 W [52], so our caps below 450 W lie outside it (measured power still follows them). The default 1000 W is the highest cap, not an uncapped state: native HPL and FP64 GEMM still hit it often.

Workloads. cuBLAS GEMMs on operands uniform on $[-1, 1)$ (BF16 and FP8 E4M3 at $32768 \times 32768 \times 6144$, FP32 accumulation); HPL (CUDA port of rocHPL) and HPL-MxP (BF16 LU, FP32 accumulation, FP64 refinement) on 8 nodes; Quantum ESPRESSO 7.6 GPU [53] (Si 1000-atom supercell, DGEMM; Au 112-atom surface with k points, ZGEMM); GPU4PySCF 1.8.1 [54], [55] (valinomycin B3LYP/def2-TZVP with density fitting; taxol RHF and DF-MP2); MFEM, SW4, LAMMPS; a Gaussian-process likelihood; MLPerf Training Llama 2 70B LoRA (NVIDIA's v5.0 FP8 code, 8 nodes, global batch 8); and LLM inference (six models).

Measurement protocol. nvidia-smi samples power, SM clock, temperature and throttle reasons on every node every 250–500 ms. Each measurement starts below 45 °C, and within a cap the variants run in ABBA order (native, zeroed, zeroed,

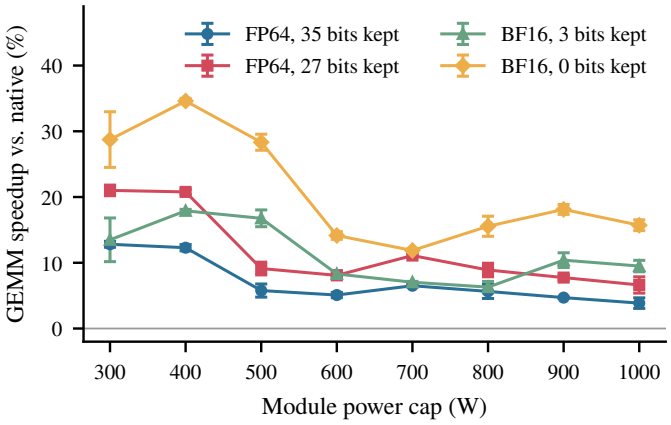


Fig. 1. GEMM speedup from mantissa zeroing under module power caps (FP64 32768×32768×2048, BF16 32768×32768×6144; 95% paired intervals, 3 nodes × ABBA).

native) on the same nodes. Windows whose recorded cap differs from the intended one are discarded (another user's tool reset the cluster's caps at least twice; affected measurements were rerun). Power-brake counters were enabled only in a diagnostic job, not in the main HPL and HPL-MxP sweeps. The first native run of a job is often slower (warm-up), so, as ABBA sequences usually start with native, the first pair can overstate the gain (Section VI-D).

Power, energy and statistics. Energy integrates the driver's 1-s averaged module power, summed over nodes. Reported power is the mean of instantaneous samples in a steady window, within $\pm 0.6\%$ of the 1-s average over 845 windows (energy avoids single readings, which deviate by -37 to $+41\%$ on bursty kernels). While capped, the 1-s average often lies below the cap, so the limiter acts on a shorter, unknown time scale. Under GEMM load the GPU draws 88–90% of module power at 900–1000 W and 77% at 400 W. Intervals are 95% t intervals of paired differences (same node, cap and ABBA round), removing the 13–17% node-to-node clock spread; only GEMM (3 nodes × 2 pairs, not fully independent, so possibly narrow) and the Quantum ESPRESSO adaptive runs (6 pairs) have them, while HPL, HPL-MxP, DF-MP2 and Gaussian-process cells have two rounds on one node set.

VI. Results: Gains Under Power Caps

A. GEMM

Zeroing speeds up GEMM at every cap, but the gain does not grow monotonically as the cap tightens (Fig. 1). FP64 keeping 27 bits is 9.1 ± 1.0, 8.1 ± 0.2, 11.1 ± 0.3 and $8.9\pm1.0\%$ faster at 500, 600, 700 and 800 W (95% intervals, 6 pairs each), a ΔW of 79–162 W; BF16 keeping 0 bits is 11.9–28.3% faster, ΔW from 121 W to above 200 W.

B. HPL and HPL-MxP on 8 Nodes

The HPL residual check determines the usable bit count, and it must be checked at several sizes (Table II). From 36 to 28 bits the residual grows 86–410×, of the order of $2^8 = 256$, so 28 bits sits less than one bit from the threshold and is not an operating point, whereas 36 bits leaves a margin of about 100× (at least 98×, about seven bits). We first tuned native HPL at 1000 W. The configuration inherited from an H200 study ($N = 262144$, NB 1024, 2×4, default NCCL) gave 374.8 TFLOP/s; fixing NCCL at 8 channels with 2 MiB buffers (+4.5%) and raising N to 286720 (+2%) gave 390.2–398.8 TFLOP/s in three runs, and every neighboring setting of N, NB, split fraction and NCCL parameters fell within this ≈2% run-to-run spread (26 settings; 4×2 grids were 14% slower). With 36 bits at the tuned configuration, HPL is 4.0–11.0% faster at 400–800 W (Fig. 2), $\Delta W = 39$–127 W (the 127 W at 800 W interpolates across the 800–1000 W gap and overestimates), and uses 4.1–9.3% less energy. At the default 1000 W cap native HPL is still capped in 75% of the samples (SM 1936 MHz) against 7% for the 36-bit run, which is therefore 3.9% faster and uses 7.5% less energy per solve. The untuned configuration gave 3.4–8.3% at 400–800 W and 0.7/7.7% at 1000 W. (28 bits is 3.8–16.3% faster, but its residual reaches 15.4 at $N = 286720$; we list it only as a boundary reference.)

HPL-MxP (BF16 LU with FP64 refinement) zeroes the BF16 operands whenever they are written back. Our runs use a test matrix whose diagonal entries equal the sum of the magnitudes of the off-diagonal entries in their row (diagonal dominance), factor the diagonal tiles approximately (`--approx-diag`), and refine with Richardson iteration and

TABLE II
HPL scaled residuals on 8 nodes at 1000 W (threshold 16; repeated runs reproduce exactly; $N = 286720$ is the tuned configuration of the cap sweep). Bits are significand bits including the hidden bit (36 bits = 35 fraction bits), rounded to nearest even. (F): fails the check; bold: the failure that rules out 28 bits.

N	Native	36 bits	28 bits	24 bits	20 bits
131072	1.5×10^{-4}	0.16	13.97	456 (F)	9510 (F)
196608	1.3×10^{-4}	0.072	**18.09 (F)**	462 (F)	2850 (F)
262144	2.6×10^{-5}	0.042	14.70	91 (F)	599 (F)
286720	7.6×10^{-5}	0.038	15.40	—	—

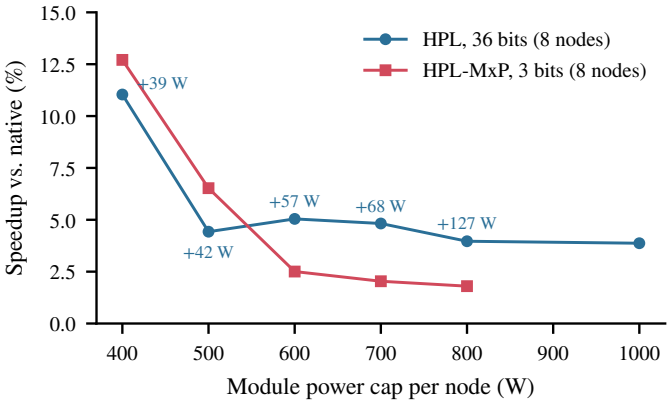


Fig. 2. 8-node HPL (36 significand bits, tuned, $N = 286720$) and HPL-MxP (3 kept bits, $N = 491520$, diagonally dominant test matrix) speedups; labels give the equivalent power saving ΔW of HPL per node.

TABLE III
DELIVERED PERFORMANCE ON 8 NODES IN TFLOP/S (MEANS OF TWO RUNS). HPL: TUNED, $N = 286720$; HPL-MXP: $N = 491520$, REPORTABLE RATE INCLUDING REFINEMENT. THE 8 GPUS' FP64 TENSOR-CORE PEAK IS ABOUT 536 TFLOP/S (8×67 TFLOP/S [52]).

	HPL (FP64)		HPL-MxP (BF16 LU)	
Cap per node	Native	36 bits	Native	3 bits
400 W	225.0	249.9	2076	2340
500 W	288.0	300.7	2823	3007
600 W	318.0	334.0	3229	3310
700 W	346.2	362.9	3449	3519
800 W	370.8	385.6	3655	3720
1000 W	393.9	409.2	3015*	3976

*Native HPL-MxP trips the chassis power brake at 1000 W.

TABLE IV
QUANTUM ESPRESSO WITH IN-PROGRAM TRUNCATION (ROUND TO NEAREST) OF DAVIDSON BLOCKS: MEAN $\pm$ 95% t INTERVAL OVER 6 INTERLEAVED PAIRS ($m = 2$: 2 RUNS); NATIVE ROWS GIVE THE MEAN OF THE 6 NATIVE RUNS.

System	Setting	Time	Energy	ΔE (Ry)	SCF
Au 112 (450 W)	Native	94.2 s	36.9 kJ	—	28
	Adapt. m=0	$-4.2 \pm 0.7\%$	$-3.9 \pm 1.1\%$	$< 10^{-8}$	28
	Adapt. m=2	-3.7%	-2.9%	$< 10^{-8}$	28
	Fixed 24	$-2.7 \pm 1.1\%$	$-2.7 \pm 1.6\%$	3×10^{-8}	28
Si 1000 (550 W)	Native	213.1 s	113.1 kJ	—	24
	Adapt. m=0	$-2.4 \pm 0.5\%$	$-2.5 \pm 0.6\%$	$< 10^{-8}$	24
	Adapt. m=2	-1.6%	-1.6%	$< 10^{-8}$	24
	Fixed 28	$-2.3 \pm 0.6\%$	$-2.3 \pm 0.6\%$	$< 10^{-8}$	24

an FP32 preconditioner. The HPL-MxP rules allow strict diagonal dominance "for testing purposes only" [56], and on such a matrix even coarse L and U factors precondition well, so the stability reported here may be a property of this matrix and cannot be assumed for the generators allowed in official runs. We use 3 kept bits (Table III lists the delivered rates of both benchmarks), stable at all four sizes $N = 245760$–491520 (residual 1.4–3.8, always 3 refinement iterations). At $N = 491520$ this is 12.7/6.5/2.5/2.0/1.8% faster at 400/500/600/700/800 W, $\Delta W = 34$–45 W (35/45/37/34/38 W; at 800 W the native curve is extended with a 900 W point, 3827 TFLOP/s, measured on the same nodes in a separate job), with 3.4–10.7% less energy. Refinement grows from 1 iteration (native) to 3 at every cap, while the speedup of the LU itself shrinks from 16.0% at 400 W to 6.0% at 800 W, so above 600 W only about 2% remains after the extra refinement. Keeping 0 bits gives 7.1–26.0% (ΔW 72–146 W) at $N = 491520$ with a residual of 10.6–11.1 (a margin of only about 1.5×), but at $N = 245760$ refinement jumps from 1 to 52 iterations and performance falls by 70%: passing the residual check does not imply being faster. With fans at full speed, native HPL-MxP at 1000 W trips the chassis hardware power brake (4–10 s each time; in the diagnostic job with counters enabled, the 0-bit version did not), so we exclude 1000 W from speedups.

C. Plane-Wave DFT and Quantum Chemistry

Plane-wave DFT is tolerant: with 28 kept bits the total energies of Quantum ESPRESSO Si 1000 and Au 112 move by 1.0×10^{-5} and 2.3×10^{-5} Ry under truncation and by 3.5×10^{-7} and 2.8×10^{-7} Ry under round-to-nearest. With nearest rounding the gold surface still converges at 20 bits (5.5×10^{-5} Ry, both runs), while for Si 1000 20 bits is the failure edge (one run converges with 1.1×10^{-4} Ry, the other aborts at iteration 24); truncation diverges or slows at 20 bits. Gaussian-basis chemistry needs more: GPU4PySCF valinomycin DFT errs by 1.1×10^{-8} E_h at 44 bits; at 36 bits the energy oscillates near 10^{-8} E_h and misses a 10^{-10} threshold, but with the chemically sufficient 10^{-7} threshold 40, 36 and 32 bits converge in the native 12 iterations (32-bit nearest error 1.2×10^{-5} E_h, ≈0.007 kcal/mol), consistent with the ≈37 significant bits that suffice for density-matrix purification [57].

Through the shim (every GEMM truncated; the baseline takes the same copy path), Si 1000 at 550 W with 28-bit nearest rounding is 6.2% faster with 6.4% less energy and Au 112 at 450 W 4.9% faster with 5.1% less energy (one run each, between baseline runs). These are upper bounds for in-library truncation; against the unmodified program the shim's ≈5% copy cost makes it 0.4% slower (Au 112, 450 W).

D. Tolerance-Driven Bit Counts in Quantum ESPRESSO

The adaptive rule truncates the wavefunction blocks before and after the Hamiltonian is applied in the Davidson diagonalization. On both systems it keeps every printed digit of the native total energy (10^{-8} Ry) and the SCF iteration count while cutting time by 2.4–4.2% and module energy by 2.5–3.9% (Table IV); on the gold surface k rises from 16 to 39. Excluding the pair with each job's slower first native run (gold surface: 96.9 s versus 94.2–94.7 s), it saves 3.9% of the time on the gold surface and 2.3% on Si 1000. The self-consistent iteration corrects errors within each Davidson step's tolerance, and the energy is second order in the wavefunction error; fixed 24 bits, coarser than the late threshold, leaves 3×10^{-8} Ry.

The gain is not only a power effect. On the gold surface the rule beats fixed 24 bits in all 6 pairs by 0.6–1.8 points, but Davidson also does about 7–9% less work (311 versus 336 Hamiltonian applications), and in a run whose cap had been externally reset to 1000 W (not power-bound, equal clocks) its SCF part was still about 9% faster than native and fixed 24 bits (13.6 versus 14.9–15.0 s): about half of the gold-surface gain comes from fewer Davidson iterations, for reasons not yet identified. On Si 1000 the work is unchanged (206 versus 205), so its 2.4% reads as a power effect; the rule beats fixed 28 bits in all 6 pairs, but only by 0.18 ± 0.14 points. Its value is that no safe bit count must be known in advance: one margin leaves both systems unchanged and matches or beats the best fixed count, found only after the fact.

E. A Power-Hungry Application: DF-MP2

Quantum ESPRESSO does not fill a 1000 W cap on average (795–828 W for Si 1000, 535–600 W for the gold surface);

TABLE V

DF-MP2 (TAXOL DEF2-TZVP): SPEED RELATIVE TO NATIVE IN TWO ROUNDS, AND THE ERROR $|\Delta E|$ OF THE MP2 CORRELATION ENERGY IN E_h (TRUNCATED VARIANTS ROUND TO NEAREST). AT 1000 W THE TRUNCATED VARIANTS RUN AT ABOUT NATIVE SPEED AND DRAW 61–70 W (32 BITS) AND 106–115 W (20 BITS) LESS, USING 7.7–8.6% AND 13.7–14.4% LESS ENERGY.

Cap	Native	32 bits	20 bits	FP32	Emu. 32
1000 W	39.5 s	+0.0/+0.6%	+0.8/+0.9%	+2.6/+11.5%	−36%
700 W	44.1 s	+5.0/+5.5%	+7.4/+7.7%	+17.8/+18.3%	−28%
500 W	52.0 s	+5.0/+4.7%	+8.6/+8.1%	+15.1/+14.8%	−18%
$\lvert\Delta E\rvert$	—	1.2–1.6×10^{-11}	1.2×10^{-8}	2.8–4.2×10^{-8}	2.2×10^{-8}

GPU4PySCF's density-fitted MP2 [54] draws more (GPU about 800 W, module 869–876 W), though at 1000 W only 24–53% of the samples hit the cap; matrix multiplication takes 78% of its GPU kernel time, nearly all of it the FP64 contraction $g_{ab} = B_i B_j^{\mathsf{T}}$ over slices of the three-index tensor B, which we zero in place once after it is built; GPU4PySCF's FP32 path and cuBLAS emulation run interleaved after one DF-RHF (taxol def2-TZVP, 2228 basis functions; two rounds per cap, agreeing within 0.6 points for the truncated variants).

Truncation changes the MP2 correlation energy by at most $1.2\times10^{-8}\,E_h$ (Table V). FP32 is faster still (its two rounds at 1000 W differ widely) and errs by only a few $10^{-8}\,E_h$, within chemical accuracy, as the tensor's exponent span is narrow (27 binades, i.e., powers of two, overall; a median of 18 within a row); cuBLAS emulation errs by $2.2\times10^{-8}\,E_h$ but is 18–36% slower than native at this shape ($m = n = 2002$, $k = 5477$).

F. FP64 Beyond GEMM

Ozaki-type emulation covers only dense GEMM, so we also measured FP64 work beyond tensor cores (Table VI). Gains shrink with the operations between truncation and multiplier and with the untruncated share of the traffic: N-body position differences regrow full mantissas, and masking them inside the loop costs more issue slots than it saves; a copy gains only when its bandwidth is power-capped; CG's traffic is dominated by vectors and untruncatable indices (it keeps its 490 iterations; true residual 4×10^{-9} at 36 bits). In MFEM, SW4 and LAMMPS we zeroed per-step coefficient arrays (about 43% and 9% of MFEM's and SW4's memory traffic; cache-resident EAM tables), so the rest of the full-mantissa computation dilutes kernel-level gains of 7–18%; unzeroed builds match the originals bitwise or within noise, and solutions change by at most 2.3×10^{-11} (MFEM) and 3.4×10^{-11}/3.4×10^{-6} (SW4 receivers, 36/20 bits; doubling its grid changes them by 4–10%). MFEM is unchanged at 1000 W.

VII. When Format Changes Win and When They Fail

Truncation keeps every element's exponent, whereas per-tensor scaled formats (FP32, FP8) need the whole tensor inside their range and Ozaki-type emulation slices each row (column) relative to its maximum, losing elements more than 2^{-k} below

TABLE VI

FP64 WORK WITHOUT TENSOR CORES: SPEEDUP WITH 36 AND 20 KEPT BITS (OWN KERNELS AT 400–700 W; THIRD-PARTY CODES, ROUNDED TO NEAREST, AT THE CAPS GIVEN), AND MODULE POWER SAVED AT 1000 W WITH 20 BITS.

Kernel or code	36 bits	20 bits	Saved
FMA loop	+2.7–4.8%	+5.6–9.0%	138 W
N-body, inputs only	+1.3–2.0%	+2.6–3.9%	59 W
N-body, also in loop	—	−7.7 to −8.7%	60 W
Memory copy (450 W)	+7.7%	+14.0%	70 W
CG, matrix values	$\leq$+1.7%		7 W
MFEM (450/550 W)	+0.9/+0.7%	+2.0/+1.4%	—
SW4 (350/450 W)	+0.6/+0.3%	+1.3/+0.5%	—
LAMMPS EAM	0.0–0.4% (noise)		—

TABLE VII

THE TWO GEMM SHAPES THAT EACH CARRY 35% OF VALINOMYCIN DFT FLOPS: TALL-SKINNY $757248\times300\times2958$ AND LONG-k $2958\times2958\times76800$ (TFLOP/S; RELATIVE ERROR IN PARENTHESES; GEMMUL8 ERRS BY 1.0×10^{-10} AND 8.2×10^{-10} ON THE TWO SHAPES, 10–80 TIMES MORE THAN TRUNCATION TO 36 BITS).

	Tall-skinny		Long k	
Method	1000 W	575 W	1000 W	575 W
FP64 native	51.6	36.8	50.6	36.2
Truncation, 36 bits (1.0×10^{-11})	54.0	38.9	53.7	38.0
cuBLAS emu., 32 bits (9.7×10^{-12})	19.5	16.6	65.4	46.0
cuBLAS emu., 28 bits (2.3×10^{-9})	24.2	20.9	89.9	65.6
GEMMul8, 10 moduli	17.9	16.7	102	75.6

it for k kept bits. Operands span 12–14 binades (central 99.9%) in HPL, about 13 in Llama 2 70B weights, 71–93 in Quantum ESPRESSO and 371–690 in valinomycin (down to 2^{-691}, beyond FP32's range): plane-wave DFT (too wide for FP16 or FP8, modest precision) has no smaller-format alternative to zeroing, Gaussian-basis chemistry needs both range and precision, and LLMs and HPL can move to smaller formats.

A. Crossover with INT8 Emulation of FP64

cuBLAS 13 emulates FP64 with a settable number of mantissa bits b in $\lceil (b+1)/8 \rceil$ 8-bit slices: 24 and 28 bits both use four (identical results, 2.3×10^{-9}), 32 and 36 bits five. Against truncated native FP64 at similar error on the same node and cap ($32768\times32768\times2048$, 400–1000 W, ABBA), three slices (5×10^{-7}) beat truncation to 20 bits by 51–73% and four slices (2×10^{-9}) beat 28 bits by 6% (1000 W) to 23% (400 W), whereas truncation to about 36 bits (10^{-11}) is 11–28% faster than five slices and over 40% faster at 10^{-13} and below. The crossover is thus near 28–32 needed bits, and on this shape lower caps favor emulation (caps on different nodes; single-node Table VIII shows the same trend).

The crossover depends on shape. Two shapes each carry 35% of the flops of the valinomycin DFT run (Table VII). With $n = 300$ emulation cannot amortize its splitting and scaling (even at 28 bits it runs at 0.47–0.57× native speed), so truncation wins at every accuracy; with $k = 76800$ emulation is 21–22% faster than truncation at 10^{-11} and the less accurate

TABLE VIII
SQUARE GEMM ($32768\times32768\times2048$) IN TFLOP/S BY CAP ON ONE NODE, NATIVE FP64 FIRST AND THE OTHER METHODS ORDERED BY RELATIVE ERROR.

Method (error)	1000 W	575 W	450 W
FP64 native	59.2	43.2	35.8
Trunc., 36 bits (1.0×10^{-11})	61.8	45.5	38.6
GEMMul8, 10 moduli (1.0×10^{-10})	70.8	51.4	34.7
cuBLAS emu., 28 bits (2.3×10^{-9})	67.8	55.2	47.5
Trunc., 28 bits (2.6×10^{-9})	63.2	46.6	39.6
FP32 SGEMM (8.1×10^{-7})	51.2	40.4	34.6
3xTF32 (4.8×10^{-6})	96.6	72.0	56.4

GEMMul8 nearly twice as fast at 10^{-9}. The best choice is thus per call, even within one application.

A shape sweep and a predictive model. With $m = 8192$ we swept $n \in \{256, 1024, 4096, 8192\}$ and $k \in \{256, 1024, 4096, 16384, 81920\}$ at 1000, 575 and 450 W (two rounds; Fig. 3). Truncation to 36 bits in the producing kernel beat native in all 60 shape–cap combinations (3.8–11.9% at 450 W, 3.6–5.9% at 575 W, 0.5–4.8% at 1000 W). cuBLAS emulation with 32 bits wins roughly when $n \geq 4096$ and $k \geq 16384$ (at best 0.74× the truncation time, at 1000 W), and its winning region shrinks at lower caps (5 of 20 shapes at 1000 and 575 W, 2 at 450 W), yet where it is not itself power-bound, lower caps favor it ($n = 8192$, $k = 4096$: time ratio 1.17, 1.08, 1.05 at 1000, 575, 450 W). The direction depends on the emulation's own power: on the square shape (Table VIII) GEMMul8 draws about 970 W at a 1000 W cap and slows to 650–720 MHz at 450 W, where truncation wins, while cuBLAS emulation with 28 bits draws only about 706 W and gains ground at lower caps. Ratios are paired on one node, but the four n ran on different nodes, so cross-shape and cross-cap trends may include node differences. GEMMul8 with 10 moduli needs only 0.47–0.80× the truncation time when $n, k \geq 4096$, at an order of magnitude larger error, and is up to 4.5× slower when n or k is small. As emulation adds input splitting ($\propto mk + nk$) and output accumulation ($\propto mn$) to the mnk multiply, we fit $T_{\mathrm{emu}}/T_{\mathrm{trunc}} = A + B\,(1/m + 1/n) + C/k$, reaching $R^2 = 0.98$–0.99 for GEMMul8 versus 0.86–0.89 for a single effective size $1/(1/m + 1/n + 1/k)$; cuBLAS emulation reaches only 0.88–0.89, no better than the single-size model. On the two held-out valinomycin shapes (two caps × two methods) the model picks the winner in 8 of 8 cases (ratio errors 6–15% for cuBLAS and 24–37% for GEMMul8; $m = 757248$ lies far outside the fitted range), but all eight lie far from the crossover (0.5–0.8× or over twice the truncation time), so this is a loose test, and with m fixed at 8192 the dependence on m is untested: a starting point for choosing per call from (m, n, k) and the cap on GH200.

B. FP32 and 3xTF32: Accumulation, Not Range

3xTF32 (our own implementation in the shim and a benchmark script, not CUTLASS's) is the fastest method in Table VIII but only FP32-accurate: its error grows almost linearly with the inner dimension (7.5–7.9× per 8× in k; 4.8×10^{-6} at $k = 2048$ on 4096×4096, 18 and 7 times that of FP64 rounded to nearest and truncated to 20 bits), whereas FP32 SGEMM grows as $\sqrt{k}$ and FP64 truncation not at all. This points to systematic error in TF32 tensor-core accumulation, consistent with the round-toward-zero accumulation found on V100, T4 and A100 [58] (bit-accurate Hopper models: [59]).

TABLE IX
REAL DATA (1000 W): FLOP-WEIGHTED FRACTION OF NONZEROS MORE THAN 2^{-32} BELOW THEIR ROW MAXIMUM, AND THE TOTAL-ENERGY ERROR OF EACH METHOD. "NEAREST": 36 OR 32 BITS ROUNDED TO NEAREST; CUBLAS EMULATION USES FIVE SLICES FOR 36 AND 32 BITS, SIX FOR 40; AUTO: ITS AUTOMATIC MODE (AT MOST 79 BITS). AU 112: FRACTION FROM A TWO-ITERATION INPUT, ERRORS FROM THE FULLY CONVERGED INPUT.

	Valinomycin	QE Si 1000	QE Au 112
Below 2^{-32}	24–44%	0.24–0.54%	0.008–0.02%
Nearest, 36 bits	$9.6\times10^{-7}\,E_h$	0 Ry	0 Ry
Nearest, 32 bits	$1.2\times10^{-5}\,E_h$	0 Ry	4×10^{-8} Ry
cuBLAS emu., 36/32	$\mathbf{5.25\times10^{-2}\,E_h}$	6×10^{-8} Ry	5×10^{-8} Ry
cuBLAS emu., 40	$8.4\times10^{-8}\,E_h$	—	—
cuBLAS emu., auto	$4\times10^{-9}\,E_h$	—	—
GEMMul8, 10 moduli	diverges	out of memory	3.7×10^{-7} Ry

Quantum ESPRESSO's operands fit in FP32's range, so we replaced every FP64 and complex FP64 GEMM with single precision through the shim (3xTF32 multiplies big and small FP32 parts three times on TF32 tensor cores; C is combined in FP64). On the gold surface at 450 W (two interleaved runs each) FP32 aborts at SCF iteration 26, and 3xTF32 and TF32 at iteration 2, as the Davidson bands fail to converge and the integrated charge drifts, while FP64 rounded to 24 and 20 bits stays within 8.1×10^{-6} and 5.5×10^{-5} Ry of native in 28 iterations; on Si 1000 at 550 W FP32 and 3xTF32 abort at iterations 14 and 9. The cause is accumulation over the plane waves (inner dimension 95386 on the gold surface plus a real GEMM with $k = 4.3\times10^6$; 375286 for Si 1000), where 3xTF32 already errs by 5.6×10^{-5} to 2.2×10^{-4}, the same order as plain TF32. FP32 GEMM on Hopper is also slower than FP64 tensor cores (an estimated 1.5× per SCF iteration on the gold surface). Replacing every GEMM with single precision thus fails for this long-inner-product solver although the range fits; FP32 in selected steps only [48] is a different matter.

C. Exponent Span Limits Format Changes

Synthetic data. On $8192\times8192\times16384$, where emulation is faster at narrow spans, we draw element exponents uniformly over S binades (1000/575 W, two rounds; Fig. 4). Truncation to 36 bits keeps a median elementwise error of 1.0–1.1×10^{-11} and a 4.4–6.3% speedup. Fixed 32-bit cuBLAS emulation degrades by 4–9× in median and 5–19× at p99 ($S = 16$–256), as if 2–3 bits were lost, which costs a whole extra slice to recover; its dynamic mode (58 to 301 bits, native FP64 beyond its maximum) takes 1.6–2.6× the truncation time, including the work of determining the span. FP32 overflows at $S = 256$; GEMMul8 is barely affected by uniform spreads.

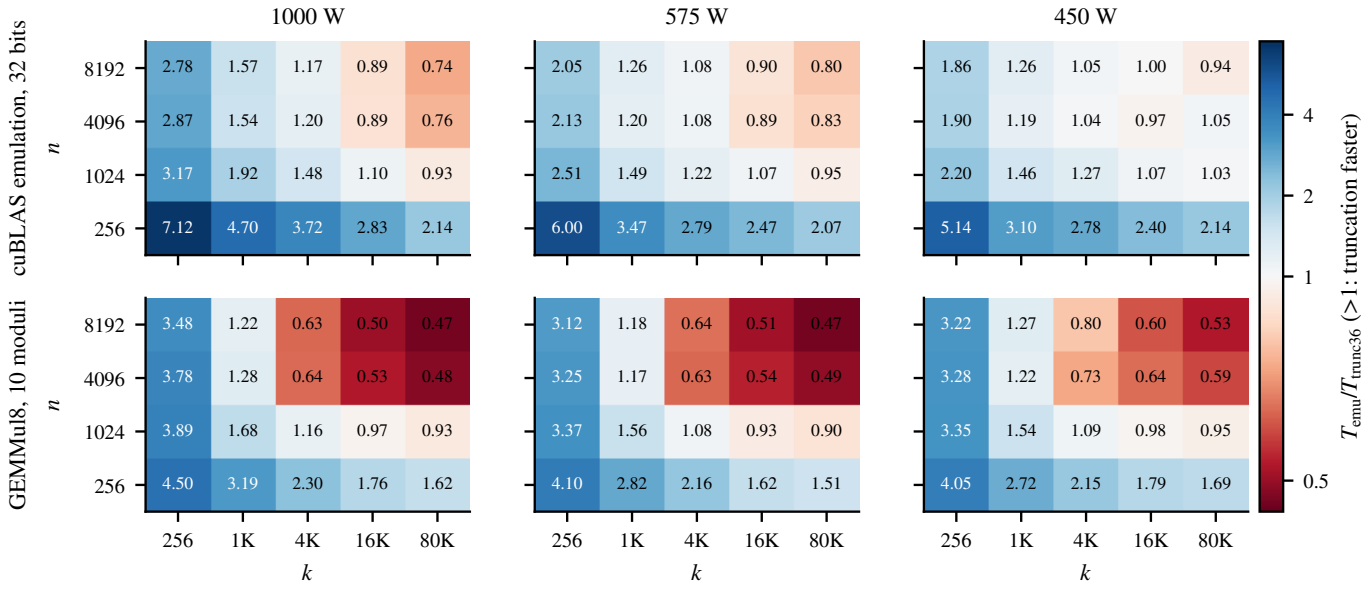


Fig. 3. Time of INT8 emulation relative to truncation to 36 bits ($m = 8192$; blue: truncation faster) for cuBLAS emulation with 32 bits (top; error comparable to truncation, $\approx 10^{-11}$) and GEMMul8 with 10 moduli (bottom; about ten times larger error); columns: module caps; means of two rounds.

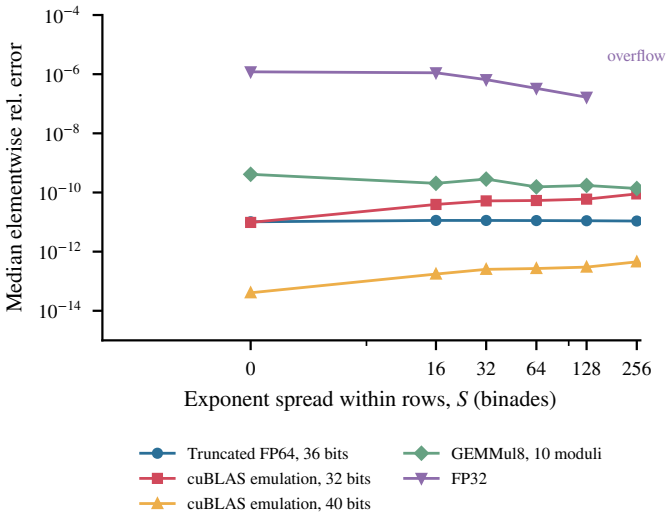


Fig. 4. Synthetic exponent spread: elements of $8192\times8192\times16384$ operands have exponents uniform over S binades within each row (1000 W).

Real data. Per GEMM the shim samples 256 rows of op(A) and 256 columns of op(B) and records the flop-weighted fraction of nonzeros more than 2^{-32} below their row maximum (validated on synthetic data). Gaussian-basis orbitals decay exponentially with distance, so in the worst valinomycin shape 87% of the elements lie that far below their row maximum. There fixed 36-bit cuBLAS emulation (five slices, like 32 bits) converges normally in 12 SCF iterations to a total energy off by 52 mE_h (≈33 kcal/mol) and GEMMul8 diverges, whereas rounding to 36 bits errs by 9.6×10^{-7} E_h (Table IX). The user-fixed bit count is what fails: with 40 bits (six slices) emulation is correct (8×10^{-8} E_h) but slower (below), and the automatic mode (at most 79 bits) is correct (4×10^{-9} E_h) but sends the 89% of flops that would need 80 or more bits to native FP64, emulates the rest with 72–79 bits and takes 2.1× the native time: not impossible, but not worthwhile. HPL is the other extreme: only 2.7×10^{-6} of its flop-weighted elements lie that far below their row maximum, and fixed 36-bit emulation passes the residual check (0.197, against 0.057 for HPL's own rounding to 36 bits) at 0.65× native speed on one GPU ($N = 81920$; automatic mode 0.35×); on Quantum ESPRESSO, with narrow per-row spans, it errs by only 5–6×10^{-8} Ry. Uniform synthetic spreads understate the problem, as in real data many elements of a row lie far below its maximum; a wide per-row span, which the shim measures cheaply beforehand (Fig. 5), is a risk indicator, necessary but not sufficient for failure, and tile-level mixed precision can also handle such data (Section VII-D).

Wide range, partly power-bound DFT. These DFT runs draw only 650–690 W at 1000 W but are power-bound below. For valinomycin (grid level 6) at 550 and 450 W, rounding to 36 bits through the shim is 3.3–3.5% faster than the copy baseline, with 2–3% less energy and an error of 9.5×10^{-7} E_h, while correct emulation with 40 bits, about ten times more accurate (8.4×10^{-8} E_h), takes 1.48–1.53× the baseline time (two rounds each). On the default grid, where FP64 GEMM takes 32% of the runtime, we estimate only 32% × 5–6.5% ≈ 1.6–2.1% at 500–800 W (GEMM speedup at 35 bits; GEMM time assumed proportional to flops at 1000 W speed). Neither number means truncation beats INT8 emulation here: choosing by shape (Table VII) bounds the gain at about 3.5% with cuBLAS emulation for the long-k shape, or 6.6% with the less accurate GEMMul8.

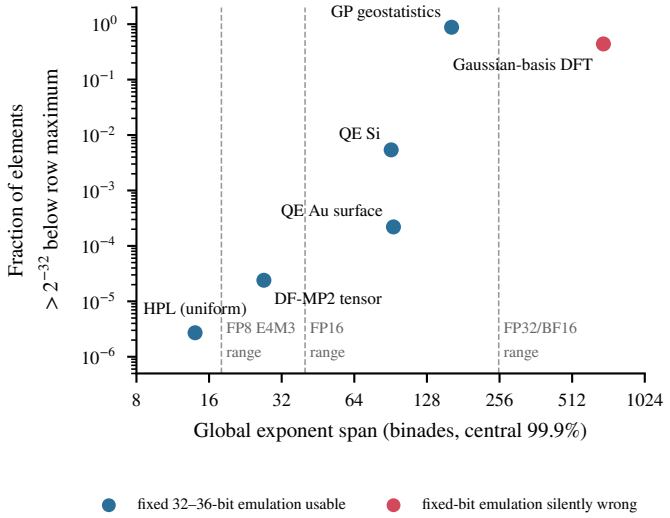


Fig. 5. Global exponent span (central 99.9%) against the flop-weighted fraction of elements more than 2^{-32} below their row maximum, with the measured outcome of fixed 32–36-bit cuBLAS emulation (e.g., HPL: residual 0.197, passes; Gaussian-basis DFT: 52 $\mathrm{mE_h}$ error; Gaussian process: span 162, estimate unchanged). All points are measured; sources in `gh200/results/range_map_points.csv`.

TABLE X
GAUSSIAN-PROCESS LIKELIHOOD ($n = 80000$, SIX RANGE PARAMETERS): LARGEST LOG-LIKELIHOOD DEVIATION FROM NATIVE AND SPEED RELATIVE TO NATIVE (TWO ROUNDS; TRUNCATION ROUNDS TO NEAREST). AT 1000 W TRUNCATION ALSO USES 2.6–4.5% (36 BITS) AND 7.2–8.7% (28 BITS) LESS ENERGY.

	Trunc. 36	Trunc. 28	cuBLAS emu. 36	FP32
$\max\lvert\Delta\ell\rvert$	2.4×10^{-5}	7.2×10^{-4}	2.2×10^{-3}	87
1000 W	+2.4/+3.2%	+3.0/+3.9%	−41%	−8.9/−8.2%
700 W	+4.9/+5.1%	+8.2/+7.6%	−31%	+3.5/+3.0%
500 W	+5.0/+4.7%	+7.7/+7.4%	−21%	+4.3/+3.6%

D. Wide Range and Power-Bound: Gaussian-Process Geostatistics

Gaussian-process maximum-likelihood estimation factors an $n\times n$ Matérn covariance matrix by Cholesky at every likelihood evaluation; as in HPL almost all work is FP64 GEMM, but the covariance decays exponentially with distance. Our ExaGeoStat-like kernel uses $n = 80000$ random 2-D locations, Matérn-3/2 covariance (nugget 10^{-4}), blocked Cholesky ($n_b = 1024$, cuBLAS DGEMM trailing updates) and six range parameters (0.02–0.08); each panel L_{21} keeps k bits in place before the trailing update in one fused pass. Native draws 953–970 W at 1000 W and is capped 91–94% of the time; the sampled panel rows span 162 binades, and 88% of panel nonzeros lie over 2^{-32} below their row maximum.

The log-likelihood is about 1.2–1.7×10^5, so the 28-bit deviation of 7.2×10^{-4} is far below what matters (Table X). At 1000 W, where native is capped 91–94% of the time, 28 bits is 3.0–3.9% faster (36 bits 2.4–3.2%), but this depends on the setting: with the Morton ordering used below ($n = 60000$; native capped in only 19–54% of the samples) 28 bits gains +0.0/+0.1%, and the fit below 1.8–2.3%. FP32's deviation makes it unusable, possibly because the covariance's conditioning worsens with n (at $n = 8000$ FP32 still finds the same estimate). cuBLAS emulation is less accurate than 28-bit truncation and slower, partly because the trailing update's inner dimension equals the block size, 1024, a shape that favors truncation (Section VII-A); despite a wider per-row span than in the DFT case it only loses accuracy instead of going wrong, so a wide span is necessary but not sufficient for failure, which also depends on what the dropped small elements contribute.

A one-parameter maximum-likelihood fit. We fitted the range parameter, with variance and smoothness fixed ($n = 60000$; 1000 and 700 W; two interleaved rounds). Truncation to 36 and 28 bits and cuBLAS emulation return the native $\hat\theta = 0.05007952$ in every printed digit, with maximized log-likelihoods within 4.8×10^{-6} (36 bits) and 2.3×10^{-4} (28 bits) of native, while FP32 moves $\hat\theta$ by 0.49% and loses 0.72. With 28 and 36 bits the fit is 7.1–7.3% and 4.8–4.9% faster at 700 W (6.6–6.8% and 4.5–4.8% less energy) and 1.8–2.3% and 1.6–2.2% faster at 1000 W (9.1–9.9% and 3.5–5.6% less energy); emulation is 32–41% slower. Of the three whole-matrix choices tested (FP32, 36-bit emulation, truncation), only truncation is both fast and accurate here.

Composing with tile mixed precision. With the locations in Morton order, we let the trailing update use FP16 inputs with FP32 accumulation on the tensor cores for tiles below each column's FP64 extent, chosen by the tile-norm rule for a required accuracy of 10^{-9} as in [47]. Over the six ranges ($n = 60000$, two ABBA rounds) this is 1.60–1.63× faster than native FP64 at 1000, 700 and 500 W, with log-likelihood deviations up to 0.043. Truncating its remaining FP64 tiles to 28 bits adds 1.9% at 1000 W and 7.0% and 6.1% at 700 and 500 W with an unchanged deviation, so truncation composes with format changes rather than competing with them.

ExaGeoStat itself. Through the shim, the MLE of ExaGeoStatCPP (cuBLAS 13.6; $N = 60000$) factorizes 3.0/4.0% faster with 36/28 bits than a same-copy baseline at 1000 W, where its GPU part is already partly power-limited, 5.0/8.0% at 600 W and 5.6/9.4% at 450 W (means of two ABBA rounds, which differ widely at 450 W, +6.9 and +11.9% with 28 bits, as the CPU shares the module cap), with log-likelihoods within 9.8×10^{-8} relative. The shim's 7–9% copy cost leaves it 0.5–3.1% slower than unmodified ExaGeoStat, so the gain needs an in-library implementation; its `--computation=mixed` option is not implemented, so we could not compare with its own mixed precision.

VIII. AI WORKLOADS: TRAINING AND INFERENCE

Training: truncating FP8 base weights is a negative result. MLPerf Training measures time to target quality (eval loss ≤ 0.925 on GovReport for Llama 2 70B LoRA). Truncating only the frozen FP8 base weights to 2 bits after loading (the rest follows the FP8 recipe [60]) made each 384 samples 0.9–1.2% faster but raised the samples to target from 3456 to 4224 with the same seed (22% slower at 575 W, 19% at 800 W; a second seed at 800 W: 3072 to 4224, +39%); 1 bit did not

converge. The first evaluation is always 0.008–0.010 higher (two seeds, three pairs), but native convergence differs by one evaluation interval between node sets (3072 versus 3456), so we read this only as evidence that FP8's 3 mantissa bits leave no room; rounding modes, decisive in [61], show no consistent order.

Inference: FP8 has already taken most of the benefit. BF16 rounded to nearest at 3 bits has FP8 E4M3's mantissa width and accuracy (GEMM error on uniform data: 3.34% versus 3.6–3.9%; truncation needs 4 bits): on six language models (6.7B–70B) WikiText-2 perplexity rises by 0.41–2.00% versus 0.45–2.23% (per-tensor) and 0.32–0.88% (per-channel) for FP8, and the two tie where FP8 is known to fail [62] (attention perplexity at 2k/8k context: +0.49/+1.06% versus +0.52/+1.08% for emulated FP8; 8k–32k key retrieval: both 100%). Yet FP8 GEMMs are about 1.7–1.9× as fast (575 W: 1.67–1.72× BF16 with 3 bits, 1.80–1.87× native BF16). Without switching to 2:4 structured sparsity or INT4 weights, zeroing on top of FP8 is the remaining way to beat FP8 on Hopper, which lacks FP4, but it trades accuracy for speed about one for one: keeping 2 bits of real Llama2-70B layer-40 activations and weights is 4.4/3.2/5.0/8.2% faster at 1000/700/575/450 W (weights only: 1.3–3.3%) while 70B perplexity rises by 3.4%. Bandwidth-bound decode gains little: at 1000 W zeroing leaves its speed unchanged on uniform data (2.6–3.0% faster at the same clock on real weights), and truncating weights to 2 or 0 bits saves only 1.5% or 5.5% of power at an accuracy cost.

IX. Discussion

Benefit under a fixed power budget. Under a fixed power budget, such as a rack limit, the equivalent power saving becomes usable compute. At 8 nodes × 700 W, tuned HPL with the safe 36 bits rises from 346.2 to 362.9 TFLOP/s (+4.8%); native HPL would need about 768 W per node, about 0.5 kW more for the cluster, to match. At the default 1000 W cap it still gains 3.9% in speed and 7.5% in energy per solve. The cost is one bitwise operation per element, fusable into the producing kernel, and no hardware.

Benchmark rules. TOP500 Linpack requires full 64-bit floating-point arithmetic and excludes lower-precision solves with iterative refinement and integer emulation of floating point [63]; HPL on truncated operands misses the full-precision requirement too. Since June 2025 HPL-MxP requires at least 16-bit floating point and allows strictly diagonally dominant matrices for testing only [56]; BF16 keeping 3 bits is effectively 12-bit, and our test matrix is not allowed for official runs. All HPL and HPL-MxP numbers here are thus research results, not official scores. MLPerf Inference's closed division allows pure mathematical quantization but forbids pruning non-zero weights [64], and zeroing is the former; whether MLPerf Training permits truncated weights depends on each round's rules.

Limitations. (i) One GH200 cluster (an earlier report has only preliminary H200 and MI350X results). (ii) The HPL residual is a deterministic but chaotic function of N (28 bits passes at $N = 131072$, 262144 and 286720, all close to the threshold, and fails at 196608), so operating points (36 bits for HPL, 3 for HPL-MxP) were validated at several sizes. (iii) HPL-MxP's strictly diagonally dominant test matrix with approximate diagonal factorization is allowed only for testing [56]; its stability at 3 kept bits may be a property of this matrix. (iv) Each scientific code ran one or two inputs, without forces, geometry optimization or larger systems. (v) Shim copies cost 3–9% (about 5% for Quantum ESPRESSO, 7–9% for ExaGeoStat), so shim results use a same-copy baseline. (vi) Nodes differ by 13–17% at equal power, so comparisons are paired on the same nodes (ABBA or seed); only GEMM and Quantum ESPRESSO have intervals, other cells two rounds on one node set, and the crossover sweep ran its four n on different nodes. (vii) The default 1000 W cap is the highest, not uncapped; the limiter's time scale is unknown; caps below 450 W lie outside the datasheet range [52]. (viii) Cooling and the chassis supply change conclusions (at low fan speed the GPU hits its thermal limit first; at full fan speed native HPL-MxP at 1000 W trips the power brake) and must be reported. (ix) Another user's tool reset the shared cluster's caps at least twice; every window checks the cap in effect. (x) Power comes from the driver's sensor, which on A100/H100 samples only part of the time [20]; our paired relative comparisons use one path and steady 10–20 s windows, but absolute energy needs an external meter. (xi) Fill controls cover only 0 and 20 kept bits, the 500 W rows one node and two runs, and the extra saving of zeros at 0 kept bits is unexplained. (xii) About half of the gold-surface Quantum ESPRESSO gain comes from fewer Davidson iterations, not power, for unidentified reasons. (xiii) Real-data emulation comparisons on Quantum ESPRESSO ran once per setting; GEMMul8 ran out of GPU memory on two applications. (xiv) The Gaussian-process results come from our ExaGeoStat-like kernel on synthetic data with a simplified mixed-precision re-implementation (FP64 storage, one FP16 GEMM per panel over the tiles that need it); ExaGeoStat itself was measured only through the shim, whose copy cost offsets the gain.

X. Conclusion

On a production GH200 cluster, when the GPU is power-bound and truncation is done inside the program, the HPC programs we tested that check their own results all gain: at 400–800 W, tuned HPL keeping 36 significand bits is 4.0–11.0% faster (39–127 W equivalent) and HPL-MxP keeping 3 bits 1.8–12.7% on a diagonally dominant test matrix; Quantum ESPRESSO with tolerance-driven bit counts is 2.4–4.2% faster at 450–550 W with every printed digit of the energies unchanged (on the gold surface about half of the gain comes from fewer Davidson iterations); power-hungry DF-MP2 keeping 20–32 bits is 4.9–8.3% faster at 500–700 W; and wide-range, power-bound Gaussian-process geostatistics keeping 28 bits is 7.4–8.2% faster at 500–700 W. At the default 1000 W cap HPL is still 3.9% faster and uses 7.5% less energy per solve. Microbenchmarks give upper bounds (FP64 GEMM keeping 27 bits: 6.6–21% at 300–1000 W; pure

data movement: 18% less module power at 1000 W), not application operating points. The gain comes mainly from fewer toggles in arithmetic units and data paths, suiting FP64 solvers with residual correction or self-consistent iteration that feed multipliers directly and in volume. Its limits are measurable: the crossover with INT8 emulation moves with shape and cap and can be estimated from (m, n, k) (8 of 8 held-out cases, all far from the crossover); with wide per-row exponent spans, fixed-width row-sliced emulation can silently go wrong, while more slices or the automatic mode are correct but slower than truncation; replacing every GEMM with FP32 or 3xTF32 fails on long inner products; kernels whose intermediates regrow full mantissas gain little; in FP8 inference, further zeroing trades accuracy for speed. Zeroing costs one bitwise operation per element, fusable into the producing kernel, needs no new hardware, and complements format changes such as tile-level mixed precision. All HPL, HPL-MxP and MLPerf numbers here are research results, not official scores.

Data and Code Availability

The mantissa shim, the in-program truncation of each application, the Slurm scripts, raw logs and summarized CSV files are in the project repository (branch `gh200-mantissa-study`); `gh200/analysis/paper_figures.py` regenerates every figure from the CSV files in `gh200/results/`, including Fig. 5, whose points and sources are listed in `gh200/results/range_map_points.csv`.

References

[1] S. Bhalachandra, B. Austin, S. Williams, and N. J. Wright, "Understanding the impact of input entropy on FPU, CPU, and GPU power," arXiv preprint arXiv:2212.08805, 2022.

[2] O. Antepara *et al.*, "Benchmark-driven models for energy analysis and attribution of GPU-accelerated supercomputing," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*, 2025, pp. 888–904.

[3] T. Gregersen, P. Patel, and E. Choukse, "Input-dependent power usage in GPUs," in *SC24-W: Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2024, pp. 1872–1877.

[4] H. He, "Strangely, matrix multiplications on GPUs run faster when given "predictable" data! [short]," Thonk From First Principles (Substack blog), https://www.thonking.ai/p/strangely-matrix-multiplications, 2024, published Apr. 29, 2024.

[5] J. Y. F. Tong, D. Nagle, and R. A. Rutenbar, "Reducing power by optimizing the necessary precision/range of floating-point arithmetic," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 8, no. 3, pp. 273–286, 2000.

[6] J. Pool, A. Lastra, and M. Singh, "Energy-precision tradeoffs in mobile graphics processing units," in *IEEE International Conference on Computer Design (ICCD)*, 2008, pp. 60–67.

[7] ——, "Precision selection for energy-efficient pixel shaders," in *ACM SIGGRAPH Symposium on High Performance Graphics (HPG)*, 2011, pp. 159–168.

[8] D. Guenther, T. Henriksson, R. Leupers, and G. Ascheid, "Mantissa-masking for energy-efficient floating-point LTE uplink MIMO baseband processing," in *Design, Automation & Test in Europe (DATE)*, 2016, pp. 1028–1029.

[9] P. V. Argade and D. N. Peroni, "Variable precision computing system," US Patent 11,455,766 B2 (Advanced Micro Devices, Inc.), filed Sep. 18, 2018, issued Sep. 27, 2022, 2022.

[10] K. Ozaki, Y. Uchino, and T. Imamura, "Ozaki scheme II: A GEMM-oriented emulation of floating-point matrix multiplication using an integer modular technique," arXiv preprint arXiv:2504.08009, 2025.

[11] Y. Uchino *et al.*, "High-performance and power-efficient emulation of matrix multiplication using INT8 matrix engines," in *SC '25 Workshops*, 2025.

[12] O. Antepara, L. Oliker, and S. Williams, "Roofline analysis of tightly-coupled CPU-GPU superchips: A study on MI300A and GH200," in *SC '25 Workshops*, 2025, pp. 1205–1216.

[13] A. Ujeniya, J. Eitzinger, G. Hager, and G. Wellein, "Architectural tradeoffs in the energy-efficient era: A comparative study of power-capping NVIDIA H100 and H200," arXiv preprint arXiv:2604.11391, 2026.

[14] J. Lucas and B. Juurlink, "ALUPower: Data dependent power consumption in GPUs," in *IEEE 24th International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2016, pp. 95–104.

[15] B. Rountree, D. H. Ahn, B. R. de Supinski, D. K. Lowenthal, and M. Schulz, "Beyond DVFS: A first look at performance under a hardware-enforced power bound," in *IEEE 26th International Parallel and Distributed Processing Symposium Workshops & PhD Forum (IPDPSW)*. IEEE, 2012, pp. 947–953.

[16] Y. Inadomi *et al.*, "Analyzing and mitigating the impact of manufacturing variability in power-constrained supercomputing," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15)*. ACM, 2015, pp. 1–12, article 78.

[17] T. Patki, D. K. Lowenthal, B. Rountree, M. Schulz, and B. R. de Supinski, "Exploring hardware overprovisioning in power-constrained, high performance computing," in *Proceedings of the 27th International ACM Conference on Supercomputing (ICS '13)*. ACM, 2013, pp. 173–182.

[18] J. You, J.-W. Chung, and M. Chowdhury, "Zeus: Understanding and optimizing GPU energy consumption of DNN training," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, 2023, pp. 119–139. [Online]. Available: https://www.usenix.org/conference/nsdi23/presentation/you

[19] M. Patrou *et al.*, "Power-capping metric evaluation for improving energy efficiency in HPC applications," in *ISC High Performance 2025 Workshops (EESP)*, ser. LNCS, 2025, pp. 231–244.

[20] Z. Yang, K. Adamek, and W. Armour, "Accurate and convenient energy measurements for GPUs: A detailed study of NVIDIA GPU's built-in power sensor," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC24)*, 2024.

[21] A. Angerd, E. Sintorn, and P. Stenström, "A framework for automated and controlled floating-point accuracy reduction in graphics applications on GPUs," *ACM Transactions on Architecture and Code Optimization*, vol. 14, no. 4, 2017, article 46.

[22] R. Jahadi and E. Atoofian, "Low-power register file for tensor cores," in *IEEE 15th International Green and Sustainable Computing Conference (IGSC)*, 2024, pp. 159–164.

[23] A. Dawson and P. D. Düben, "rpe v5: An emulator for reduced floating-point precision in large numerical simulations," *Geoscientific Model Development*, vol. 10, pp. 2221–2230, 2017.

[24] N. J. Higham and S. Pranesh, "Simulating low precision floating-point arithmetic," *SIAM Journal on Scientific Computing*, vol. 41, no. 5, pp. C585–C602, 2019.

[25] N. Chatterjee *et al.*, "Architecting an energy-efficient DRAM system for GPUs," in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2017, pp. 73–84.

[26] M. O'Connor *et al.*, "Fine-grained DRAM: Energy-efficient DRAM for extreme bandwidth systems," in *IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2017, pp. 41–54.

[27] S. Ghose *et al.*, "What your DRAM power models are not telling you: Lessons from a detailed experimental study," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 2, no. 3, 2018.

[28] NVIDIA, "Performance benchmarking — NVIDIA TensorRT (GPU power consumption and power throttling)," https://docs.nvidia.com/deeplearning/tensorrt/latest/performance/benchmarking.html, accessed Sep. 30, 2026.

[29] G. Pekhimenko, E. Bolotin, N. Vijaykumar, O. Mutlu, T. C. Mowry, and S. W. Keckler, "A case for toggle-aware compression for GPU systems," in *IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2016, pp. 188–200.

[30] C. Rubio-González *et al.*, "Precimonious: Tuning assistant for floating-point precision," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis (SC13)*, 2013.

[31] N. J. Higham and T. Mary, "Mixed precision algorithms in numerical linear algebra," *Acta Numerica*, vol. 31, pp. 347–414, 2022.

[32] A. C. I. Malossi *et al.*, "The transprecision computing paradigm: Concept, design, and applications," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2018, pp. 1105–1110.

[33] T. Grützmacher, T. Cojean, G. Flegar, H. Anzt, and E. S. Quintana-Ortí, "Acceleration of PageRank with customized precision based on mantissa segmentation," *ACM Transactions on Parallel Computing*, vol. 7, no. 1, pp. 1–19, 2020.

[34] A. Haidar, S. Tomov, J. Dongarra, and N. J. Higham, "Harnessing GPU tensor cores for fast FP16 arithmetic to speed up mixed-precision iterative refinement solvers," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC18)*, 2018, pp. 603–613.

[35] J. Dongarra and P. Luszczek, "HPL-MxP benchmark: Mixed-precision algorithms, iterative refinement, and scalable data generation," *The International Journal of High Performance Computing Applications*, vol. 40, no. 1, pp. 52–62, 2026, published online Sep. 25, 2025; arXiv:2509.19618.

[36] M. Klöwer, M. Razinger, J. J. Dominguez, P. D. Düben, and T. N. Palmer, "Compressing atmospheric data into its real information content," *Nature Computational Science*, vol. 1, no. 11, pp. 713–724, 2021.

[37] K. Ozaki, T. Ogita, S. Oishi, and S. M. Rump, "Error-free transformations of matrix multiplication by using fast routines of matrix multiplication and its applications," *Numerical Algorithms*, vol. 59, no. 1, pp. 95–118, 2012.

[38] H. Ootomo, K. Ozaki, and R. Yokota, "DGEMM on integer matrix multiplication unit," *The International Journal of High Performance Computing Applications*, vol. 38, no. 4, pp. 297–313, 2024.

[39] NVIDIA, "cuBLAS library documentation, section 1.5: Floating point emulation (v13.4; archived 13.2.1)," https://docs.nvidia.com/cuda/cublas/index.html#floating-point-emulation, accessed Oct. 5, 2026.

[40] A. Schwarz *et al.*, "Guaranteed DGEMM accuracy while using reduced precision tensor cores through extensions of the Ozaki scheme," in *Proceedings of the Supercomputing Asia and International Conference on High Performance Computing in Asia Pacific Region (SCA/HPCAsia)*, 2026, pp. 91–101.

[41] S. Abdulah, H. Ltaief, Y. Sun, M. G. Genton, and D. E. Keyes, "ExaGeoStat: A high performance unified software for geostatistics on manycore systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 29, no. 12, pp. 2771–2784, 2018.

[42] ——, "Geostatistical modeling and prediction using mixed precision tile Cholesky factorization," in *IEEE 26th International Conference on High Performance Computing, Data, and Analytics (HiPC)*, 2019, pp. 152–162.

[43] S. Abdulah *et al.*, "Accelerating geostatistical modeling and prediction with mixed-precision computations: A high-productivity approach with PaRSEC," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 4, pp. 964–976, 2022.

[44] Q. Cao *et al.*, "Reshaping geostatistical modeling and prediction for extreme-scale environmental applications," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022, pp. 1–12.

[45] J. Ren, H. Ltaief, S. Abdulah, and D. E. Keyes, "Accelerating mixed-precision out-of-core Cholesky factorization with static task scheduling," in *ISC High Performance 2025 Research Paper Proceedings (40th International Conference)*, 2025, pp. 1–12.

[46] S. Abdulah *et al.*, "Boosting earth system model outputs and saving PetaBytes in their storage using exascale climate emulators," in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2024, pp. 1–12.

[47] Q. Cao, S. Abdulah, H. Ltaief, M. G. Genton, D. Keyes, and G. Bosilca, "Reducing data motion and energy consumption of geospatial modeling applications using automated precision conversion," in *IEEE International Conference on Cluster Computing (CLUSTER)*, 2023, pp. 330–342.

[48] J. Woo, S. Kim, and W. Y. Kim, "Dynamic precision approach for accelerating large-scale eigenvalue solvers in electronic structure calculations on graphics processing units," *Journal of Chemical Theory and Computation*, vol. 19, no. 5, pp. 1457–1465, 2023.

[49] H. Huang, W. Shao, and J. Hammond, "Accelerating density fitting with adaptive-precision and 8-bit integer on AI accelerators," arXiv preprint arXiv:2601.08077, 2026.

[50] R. Olivares-Amaya, M. A. Watson, R. G. Edgar, L. Vogt, Y. Shao, and A. Aspuru-Guzik, "Accelerating correlated quantum chemistry calculations using graphical processing units and a mixed precision matrix multiplication library," *Journal of Chemical Theory and Computation*, vol. 6, no. 1, pp. 135–144, 2010.

[51] M. Langhammer, "Low power optimizations for a floating point multiplier," US Patent 9,606,608 B1 (Altera Corporation), 2017.

[52] NVIDIA, "NVIDIA GH200 Grace Hopper superchip datasheet," Document 3773000, linked from https://www.nvidia.com/en-us/data-center/grace-hopper-superchip/, Mar. 2025, accessed Oct. 5, 2026.

[53] P. Giannozzi *et al.*, "Quantum ESPRESSO toward the exascale," *The Journal of Chemical Physics*, vol. 152, p. 154105, 2020.

[54] X. Wu *et al.*, "Enhancing GPU-acceleration in the Python-based simulations of chemistry frameworks," *WIREs Computational Molecular Science*, vol. 15, no. 2, p. e70008, 2025.

[55] R. Li, Q. Sun, X. Zhang, and G. K.-L. Chan, "Introducing GPU acceleration into the Python-based simulations of chemistry framework," *The Journal of Physical Chemistry A*, vol. 129, no. 5, pp. 1459–1468, 2025.

[56] HPL-MxP, "HPL-MxP rules," https://hpl-mxp.org/rules.md, accessed Oct. 5, 2026.

[57] W. Dawson, K. Ozaki, J. Domke, and T. Nakajima, "Reducing numerical precision requirements in quantum chemistry calculations," *Journal of Chemical Theory and Computation*, vol. 20, no. 24, pp. 10 826–10 837, 2024.

[58] M. Fasi, N. J. Higham, M. Mikaitis, and S. Pranesh, "Numerical behavior of NVIDIA tensor cores," *PeerJ Computer Science*, vol. 7, p. e330, 2021.

[59] F. A. Khattak and M. Mikaitis, "Accurate models of NVIDIA tensor cores," arXiv preprint arXiv:2512.07004, 2025.

[60] P. Micikevicius, D. Stosic *et al.*, "FP8 formats for deep learning," arXiv preprint arXiv:2209.05433, 2022.

[61] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep learning with limited numerical precision," in *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 37, 2015, pp. 1737–1746.

[62] J. Zhang, H. Huang, P. Zhang, J. Wei, J. Zhu, and J. Chen, "SageAttention2: Efficient attention with thorough outlier smoothing and per-thread INT4 quantization," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025, arXiv:2411.10958.

[63] TOP500.org, "The Linpack benchmark," https://www.top500.org/project/linpack/, accessed Oct. 5, 2026.

[64] MLCommons, "MLPerf inference rules, section 8.1: Weight definition and quantization," https://github.com/mlcommons/inference_policies/blob/master/inference_rules.adoc.